



Programmierpraktikum (PPR)

WiSe 2025 / 2026
Abschlussprojekt

Alexander Mehler

Giuseppe Abrami

23.02.2026

MULTIMODAL PARLIAMENT EXPLORER

Einleitung

Die Abschlussaufgabe im Programmierpraktikum im WiSe 2025 / 2026 besteht darin, einen MULTIMODAL PARLIAMENT EXPLORER gemeinschaftlich als Gruppenprojekt zu entwickeln. Zur Lösung dieses Gruppenprojekts haben sich die Teilnehmer*innen in Gruppen mit je 3 bis 4 Personen zusammengefunden. Die bisherigen Übungen dienten dem Zweck, die Studierenden auf die Erledigung dieser Gruppenaufgabe vorzubereiten. In der Gruppenarbeit wird sichtbar, wie wichtig die Nutzung einer Projekt- und Quellcode-Verwaltung ist. Hierzu verwenden wir, wie auch schon in den Übungen, GitLab¹.

Kompensation der Gruppengröße

Die nachstehenden Aufgaben sind unabhängig der Gruppengröße zu bearbeiten. Allerdings werden Gruppen, die weniger als 4 Mitglieder haben, auf die erreichten Punkte einen Faktor erhalten, um fehlende Teammitglieder zu kompensieren.

Der MULTIMODAL PARLIAMENT EXPLORER setzt sich inhaltlich aus den einzelnen Aufgaben der Einzelübungen zusammen und erweitert diese, gemäß der Praktikumsthematik.

Vorbereitungen

Jede Gruppe erstellt zur Bearbeitung der Abschlussaufgabe ein neues *privates* Projekt im PPR-GitLab und nennt dieses **Multimodal_Parliament_Explorer_X_N**, wobei „X_N“ für Ihren Gruppennamen steht². Im Anschluss ist dieses Projekt **allen** Gruppenmitgliedern sowie Ihrer Tutorin / Ihrem Tutor freizugeben, sodass alle damit arbeiten können.

¹<https://ppr.gitlab.texttechnologylab.org>

²Gruppe 1_Montag_3 wäre dann Parliament_Explorer_1_3!

Zugangsdaten Datenbank

Ihre Zugangsdaten für ihre Gruppendatenbank werden in Ihr GitLab-Projekt, durch die Praktikumsleitung, kopiert. **Bitte legen Sie Ihr Projekt daher frühzeitig an.**

Vorbemerkungen

Grundsätzlich sind alle Ergebnisse ausführlich zu dokumentieren. Bezüglich der Dokumentation beachten Sie bitte die dazugehörige Aufgabe 4. Die Dokumentationsqualität fließt, wie auch in den vorherigen Übungen, in die Bewertung mit ein. Die Verwendung anderer als die angegebenen Programmiersprachen, Bibliotheken und Werkzeuge ist nicht erlaubt. Im Rahmen dieser Gruppenaufgabe verwenden wir **Java in der Version 21**. Die Implementierung hat ausschließlich in Java zu erfolgen und das Verwenden fertiger Frameworks ist **nicht** gestattet. Der Austausch von Quellcode, die Verwendung von Lösungen bzw. *Programmschnipseln* aus dem Internet, sowie die Verwendung von Sprachmodellen jeglicher Art (z.B. ChatGPT, o.ä.) ist nicht gestattet.

Diese Übung ist in **Gruppenarbeit** zu absolvieren, jedoch ist die **individuelle Einzelleistung** kenntlich zu machen (vgl. Aufgabe 4). Hierzu gehört auch die *selbstständige* und *eigenständige* Nutzung von GitLab.

Individuelle Einzelleistung...

... bedeutet, dass sichtbar und erkennbar dokumentiert sein muss, welchen Beitrag jede*r Teilnehmende der Gruppe zum Projekt geleistet hat. Jede*r Teilnehmende wird auch nur für den geleisteten Beitrag bewertet. Der jeweilige Verdienst am Projektfortschritt bzw. am Projektergebnis fließt anteilig ein.

Verwendung eigener Arbeiten

Sie dürfen die eigenen Vorarbeiten, welche die Gruppenteilnehmer in den Einzel-Übungen (1-4) sowie in der DUUI-Wahlpflichtübung, erzeugt haben, im Abschlussprojekt nachnutzen.

Bitte beachten Sie jedoch, dass Sie die Nachnutzung kenntlich machen und Transparent dokumentieren.

NLP-Verarbeitung

Im Rahmen der Gruppenabschlussaufgabe sind von Ihnen NLP-Methoden mittels DUUI anzuwenden. Da NLP-Prozesse immer eine gewisse Zeit benötigen, müssen Sie nicht die kompletten Reden mittels NLP verarbeiten. Sie bekommen von der Praktikumsleitung eine Reihe von annotierten Reden zur Verfügung gestellt; diese müssen Sie nur noch einlesen.

Reden, die jedoch fehlen, müssen Sie noch verarbeiten.

Die DUUI-Images sowie die Remote-Dienste finden Sie im Anhang.

Multimodal Parliament Explorer

Sie haben in den letzten Wochen und Monaten eine Reihe von Applikationen und Lösungen entwickelt, welche unserem Arbeitskreis **DigitalBundestag** sehr gut gefallen haben. Aus diesem Grunde haben wir entschieden, die gesamte Thematik auf ein ganz neues Level zu bringen. Zu diesem Zweck stellen wir ein Team aus einzigartigen Entwicklern zusammen, welches die nachstehende Aufgabe meistern soll:

Entwickeln Sie als Team den MULTIMODAL PARLIAMENT EXPLORER bestehend aus einer Client- sowie Server Komponente unter Verwendung verschiedener, in der weiteren Beschreibung aufgeführten Bibliotheken und Konzepte. Der MULTIMODAL PARLIAMENT EXPLORER soll über die folgenden Funktionalitäten verfügen:

1. Automatisches Downloaden, Einlesen und Importieren von Parlamentsdebatten des Deutschen Bundestages der verschiedenen Legislaturperiode im XML-Format, sowie der Videos der einzelnen **Redebeiträgen**.
2. Analysieren, Aufbereiten und datenbankgestütztes Serialisieren der abgefragten Parlamentsreden.
3. Abbildung der Parlamentsdebatten sowie der Abgeordneten-Stammdaten, gemäß Aufgabenbeschreibung.
4. Verarbeitung der Parlamentsreden mittels verschiedener NLP-Verfahren, sowie die Nachnutzbarmachung mittels UIMA-Serialisierung.
5. Datenbankgestütztes Abfragen und Auswerten der Ergebnisse.
6. Entwicklung einer REST-API.
7. Implementierung eines RESTful Webservice zur Nutzung und Visualisierung der Parlamentsreden für das Frontend.
8. Implementierung einer komplexen Visualisierungskomponente unter Verwendung von **d3.js^a** gemäß Aufgabe 3.1.
9. Serialisierung in $\text{\LaTeX}$ (vgl. 3.2).

Das MULTIMODAL PARLIAMENT EXPLORER-Backend soll als RESTful-Webservice unter Verwendung von **Javalin** implementiert werden. Dieses Backend stellt dem Frontend alle Methoden und Funktionen zur Verfügung, um mit den Parlamentsprotokollen gemäß der Aufgabenbeschreibung zu interagieren. Zum Testen und Dokumentieren der REST-Abfragen verwenden Sie **swagger.io**, welches Sie *lokal* verfügbar machen. Das Backend ist nach objektorientierten Standards in Java zu erstellen, wie bereits erprobt. Die Verwendung von Interfaces bzw. abstrakten Klassen ist obligatorisch und zwingend erforderlich. Sorgen Sie in allen Komponenten für eine möglichst hohe Datenkapselung und vermeiden Sie wiederkehrenden bzw. duplizierten Quellcode.

Betrachten Sie die Parlamentsdebatten basierend auf den *Reden*; modellieren Sie die Datenstrukturen dazu so, dass die Reden die zentralen Bestandteile sind: Jede Rede gehört zu einer Sitzung; jede Rede hat Kommentare; jede Rede hat einen Redner, etc. Für Redner ist eine eigene Datenstruktur zu definieren, die es erlaubt, komplexere Inhalte abzubilden. Speichern Sie die Parlaments-Daten in der Ihnen zur Verfügung stehenden Datenbank ab und analysieren Sie

die Daten mit NLP-Methoden zwecks der zu realisierenden Visualisierungen. Visualisieren Sie Ihre Ergebnisse mittels **d3.js** gemäß der in Aufgabe 3.1 definierten Anforderungen. Da, gemäß Projektname, Explorieren auch die Möglichkeit der Nachnutzung impliziert, ermöglichen Sie bitte die Exportmöglichkeiten der Reden nach verschiedenen Kriterien in ein $\text{\LaTeX}$ -Format (Aufgabe 3.2). Dokumentieren Sie Ihr Projekt sowie die Projektbearbeitung und dessen Verlauf ausführlich (Aufgabe 4) und präsentieren Sie Ihr Gesamtergebnis als Gruppe im vor dem Arbeitskreis^b.

^a<https://d3js.org/>

^bDies ist dann das Tutorium

Fragen...

...zur Umsetzung stellen Sie bitte frühzeitig, damit diese schon in Ihre Projektplanung einfließen können.

Hierfür findet am **Mittwoch, 25.02.2026, 10.00 Uhr**, eine allgemeine Fragestunde via Zoom statt.

Aufgaben

Zur Durchführung lesen Sie zunächst alle Aufgaben gründlich durch, damit Sie Zusammenhänge und Abhängigkeiten nicht übersehen.

1 Projektplanung und Design

10 Punkte

Erstellen Sie alle nachfolgenden Diagramme unter Verwendung von geeigneten Softwarelösungen (z.B. $\text{\LaTeX}$)!

- (a) Definieren Sie einen Zeitplan für ihr Projekt, der auch aufzeigt, wer in Ihrer Gruppe welche Aufgaben übernimmt bzw. übernommen hat. Dokumentieren Sie dies anhand eines *Gantt*-Diagramms mit Milestones und Aufgabenpaketen. Das Projektende ist der **22.03.2026**. Die Milestones sind auch im GitLab-Projekt zu hinterlegen und zu pflegen. Der Projektverlauf ist einzuhalten und evtl. Abweichungen sind zu dokumentieren. **Die kontinuierliche sowie eigenständige Nutzung von GitLab ist daher obligatorisch.**
- (b) Nutzen Sie zur Delegation und Planung von Aufgaben, dem Tracken von Issues und Fehlern, **Tickets** in GitLab.
- (c) Erstellen Sie ein Package- und Klassen- sowie ein Use-Case-Diagramm für den MULTIMODAL PARLIAMENT EXPLORER.
- (d) Erstellen Sie ein Mock-up für Ihre Visualisierungen sowie das User-Interface.

2 Backend

12 Punkte

Entwickeln Sie das Backend Ihrer Applikation als RESTful Java-Applikation³ und implementieren Sie die nachstehenden Anforderungen zur Abbildung der Datenstrukturen. Dies ist ggf. durch die Anforderungen in den Aufgaben 3.1–3.2 zu ergänzen. Ihr Backend ist das Rückgrat Ihrer gesamten Applikation und beinhaltet alle Methoden und Funktionen für die Aufbereitung, Analyse, Verwaltung und Formatierung der Daten.

- (a) Implementieren Sie eine Datenstruktur basierend auf den in Aufgabe 1 definierten Diagrammen. Verwenden Sie hierzu objektorientierte Konzepte der Programmiersprache Java (Interfaces / abstrakte Klassen sowie deren Implementierungen).
- (b) Implementieren Sie eine parametergestützte Klasse `MongoDatabaseHandler`⁴, welche die Kommunikation mittels MongoDB gewährleistet. Dies beinhaltet die Operationen *Erstellen*, *Lesen*, *Updaten*, *Löschen*, *Aggregieren* und *Zählen*.
- (c) Die Repräsentation der Zugangs- bzw. Konfigurationsdaten hat mittels Klassenrepräsentation zu erfolgen.

Die Nutzung von Hilfsbibliotheken, welche die Datenstrukturen selbstständig in eine Datenbank abbilden, ist nicht gestattet.

- (d) Laden Sie die definierten Parlamentstexte direkt vom Bundestag⁵ herunter (via Java⁶) und importieren Sie die Protokolle in Ihre Datenbank. Eine Routine soll vorab prüfen, ob die Debatten bereits vorhanden sind.

³Die Verwendung von Maven ist verpflichtend!

⁴Die Zugangsdaten bekommen die Gruppen in ihre GitLab-Projekte kopiert.

⁵<https://www.bundestag.de/services/opendata>

⁶Die Bibliothek *Jsoup* bietet sich hier an.

- Über einen Timer / Thread soll die Abfrage neuer, noch nicht bestehender Protokolle erfolgen.
- (e) Bilden Sie die Abgeordneten der jeweiligen Wahlperiode ab und erweitern Sie die Abfragen über die Parlamentsreden hinaus:
1. Alle Abgeordneten inkl. ihrer Meta-Stammdaten sind in einer eigenen MongoDB-Collection abzulegen.
 2. Fügen Sie, falls vorhanden, die Fotos der Abgeordneten ein. Verwenden Sie hierfür die Bilddatenbank des Bundestages⁷. Die Abfragen sind in Java zu implementieren und während des Parsens zu erfassen. Bitte beachten Sie, dass jedes Bild eine Menge von Meta-Daten hat, die ebenfalls abzubilden sind.
- (f) Ermöglichen Sie ebenfalls die automatische Erfassung sowie den Download der **Video**-Beiträge⁸ pro **Rede** und serialisieren Sie diese gemäß Aufgabe 2.1 in der Datenbank.
- (g) RESTful Webservice
- (1) Implementieren Sie eine RESTful-Schnittstelle basierend auf Javalin⁹ und definieren Sie *Routen* und *Parameter* gemäß den Aufgaben 3.1 bis 3.2.
 - (2) Dokumentieren Sie die REST-Methoden und machen Sie diese unter Zuhilfenahme von *swagger.io*¹⁰ verfügbar.

2.1 Natural Language Processing

15 Punkte

- (1) Implementieren Sie Klassen und Methoden zur objektorientierten Vorverarbeitung von Parlamentsreden unter Verwendung des DOCKER UNIFIED UIMA INTERFACE (Leonhardt u. a. 2023).
- (2) Implementieren Sie Ihre Applikation so, dass alle eingelesenen Debatten mittels NLP-Methoden unter Verwendung der DUUI-Architektur vorverarbeitet werden können. Verwenden Sie die nachstehenden Vorverarbeitungsmethoden, welche als DUUI-Komponenten vorliegen:
 - Token, Sentences, POS, Dependency, Named Entity Recognition, Lemma (mittels **spaCy**)
 - Topic (mittels **ParlBERT-v2**)
 - Sentiment (mittels **GerVaderSentiment**)
 - Sarkasmus (mittels **multilingual-sarcasm-detector**)
 - Transkription der Videos (mittels **WhisperX**)
- (3) Aufgrund der NLP-Verarbeitung ordnen Sie Kommentare konkreten Abgeordneten und/oder Fraktionen zu.
- (4) Speichern Sie die Ergebnisse in geeigneter Form zur späteren Abfrage in Ihrer Datenbank und serialisieren Sie das gesamte Ergebnis der Analyse. Achten Sie darauf, dass Sie alle NLP-Analysen, außer WhisperX, auch auf die Transkription anwenden.
- (5) Erstellen Sie ein eigenes *UIMA TypeSystem*¹¹ zur Darstellung und Abbildung der Plenarreden und bilden Sie die eingelesenen Reden hierauf ab. Serialisieren Sie die NLP-verarbeiteten Reden mit diesen Informationen (Abrami und Mehler 2024)

⁷<https://bilddatenbank.bundestag.de>

⁸ Suchen Sie sich einen Tagesordnungspunkt mit einer nicht trivialen Menge an Reden, *eines* Protokolls heraus und laden Sie lediglich diese Videos automatisch herunter.

⁹<https://javalin.io/>

¹⁰<https://swagger.io/>

¹¹[Beispiel](#) • [Anleitung](#)

Implementieren Sie einen eigenen **DUUI-Reader** für die NLP-Verarbeitung der (multi-modalen) Parlamentsdebatten unter Verwendung des Ansatzes von (Hammerla, Mehler und Abrami 2025). Hierbei geht es darum, dass Sie die einzelnen Reden nicht einzeln mittels DUUI verarbeiten müssen, sondern DUUI *direkt* die Reden und die dazugehörigen Videos erfassen kann und durch Nutzen einer DUUI-Pipeline ein fertiges CAS-Objekt erzeugt, welches Sie dann nutzen können. Hierbei wird das in (5) erstellte TypeSystem angewendet werden. Implementieren Sie passende DUUI-Parameter.

3 Frontend

Implementieren Sie das gesamte Frontend unter Verwendung der *FreeMarker*-Template-Engine.

Darüber hinaus dürfen Sie für das Frontend die folgenden Bibliotheken verwenden:

- d3.js
- jQuery
- Font Awesome¹²

Über folgende Funktionen / Features soll Ihr Frontend verfügen:

3.1 Visualisierung, Abfrage und Interaktion von und mit Parlamentsdebatten 10 Punkte

- (a) Implementieren Sie die nachfolgenden *Visualisierungen* unter Verwendung der Visualisierungsbibliothek *d3.js*:
 - (1) Verteilung der Topics als *Radar chart*.
 - (2) Verteilung der POS als *Sunburst*.
 - (3) Verteilung der Sentiments als *Line chart*.
 - (4) Verteilung der *Named Entities* als horizontale *Bar chart*.
- (b) Alle Visualisierungen sind abhängig von verschiedenen Kriterien einzuschränken:
 - für alle Reden (als Standard-Anzeige)
 - für eine einzelne ausgewählte Rede
 - für eine Menge von Reden, wobei entsprechende Selektionsmöglichkeiten¹³ zu implementieren sind, die auch im Konjunktiv genutzt werden können
 - für Reden zu einem auszuwählenden Topic gemäß der NLP-Analyse (vgl. Aufgabe 2.1)
- (c) Implementieren Sie eine generische Suchfunktion, um Reden zu durchsuchen, sowie eine passende Navigation.
- (d) Implementieren Sie eine Volltext-Visualisierung für eine Parlaments-Rede, die folgende Informationen abbildet:
 1. Alle im Text gefundenen Named Entities sollen farblich hinterlegt werden, und eine Legende kennzeichnet den Typ der Annotation (also *Person*, *Ort*, *Organisation*). Die Darstellung der Annotationen soll *getoggelt* werden können.
 2. Darüber hinaus soll der ermittelte Sarkasmus auf Satzebene dargestellt werden können. Die Anzeige soll über einen Schwellwertregler modifiziert werden können.

¹²<https://fontawesome.com/>

¹³Zeitraum, Zeitpunkt, Fraktion

3. Der Sentimentwert eines Satzes ist mit einem farblichen Hintergrund darzustellen.
 4. Der Sprecher*innenname, Informationen zu seiner/Ihrer Fraktionszugehörigkeit sowie ein dazugehöriges Foto werden zu Beginn der Rede angezeigt.
 5. Visualisieren Sie die Kommentare an den dazugehörigen Stellen im Text und kennzeichnen Sie den Autor des Kommentars (Sprecher, Fraktion) durch ein Icon.
- (e) Sofern für eine Rede ein mittels NLP verarbeitetes Video vorhanden ist, soll das Video neben der stenographierten Rede eingebunden werden und abspielbar sein. Gemäß der Sentiment-Annotationen soll der jeweils gesprochene Satz in einer Sentiment-Ampel (Grün, Grau, Rot) verdeutlicht werden.

Bonus II

5 Punkte

Visualisieren Sie Ähnlichkeiten (*Text-Similarity*) zwischen Parlamentsreden, indem Sie hierfür eine neue DUUI-Komponente entwickeln, unter Nachnutzung eines fertigen Modells^a. Erstellen Sie ein Docker-Image für die DUUI-Komponente, führen Sie eine Analyse auf allen Parlamentsreden durch und visualisieren Sie die Ähnlichkeit in einer geeigneten Netzwerk-Visualisierung (mit d3.js), indem Sie dies in Ihre Applikation integrieren. Die Nutzung geeigneter, zu implementierender DUUI-Writer bietet sich hierzu an. Das DUUI-Image pushen Sie auch in die Docker-Registry des Programmierpraktikums^b.

Finalisieren Sie das Image bis zur Hälfte der Laufzeit des Abschlussprojektes, sofern Sie die Verarbeitung auf dem Server für das PPR durchführen lassen wollen.

- (a) Visualisieren Sie im Rahmen von Aufgabe 3.1 die Ähnlichkeit zwischen den verschiedenen Reden und wählen Sie hierzu eine geeignete d3.js-Visualisierung aus. Die Visualisierung binden Sie in Ihr Frontend ein.

^ahttps://python.langchain.com/docs/integrations/text_embedding/spacy_embedding/

^bdocker.ppr.texttechnologylab.org

3.2 Export und Präsentation der Parlamentsdebatten

13 Punkte

Die Präsentation sowie der Export von Parlamentsprotokollen ist ein wichtiger Bestandteil Ihrer Applikation. Sie ermöglichen damit unter anderem, die bestehenden Protokolle informativ und ansprechend auszugeben und nachzunutzen. Erstellen Sie hierzu die nachfolgenden Funktionen:

- (a) Implementieren Sie einen Reden-Exporter in ein PDF, unter Verwendung von L^AT_EX, basierend auf Ihren erfassten und verarbeiteten Reden, der die folgenden Bestandteile aufweist:
- Tagesordnung als Inhaltsverzeichnis.
 - Die einzelnen Reden...
 - ...besitzen Namen und die Fraktion des Redners/der Rednerin (sowie optional ein Bild).
 - ...besitzen statistische Angaben der relevanten NLP-Informationen. Diese sollen *optional* mittels TikZ¹⁴ grafisch visualisiert werden.
 - ...haben visualisierte Kommentare und Zurufe an den entsprechenden Stellen im Text.
 - Die Generierung hat hierbei *ausschließlich* auf den in UIMA annotierten Informationen zu erfolgen.

¹⁴<https://tikz.dev/>

- Die Auswahl der Protokolle kann für ein einzelnes Protokoll, mehrere bzw. spezifische Protokolle sowie für alle Protokolle auf einmal erfolgen. Erfolgt die Generierung für mehr als ein Protokoll, ist jedes Protokoll gemäß Sitzungs-Index und -Datum bzw. durch einen geeigneten Separator in Form eines *Kapitels* (L^AT_EX `\Chapter`) zu trennen.
 - Darüber hinaus ist die Generierung pro Redner, pro Fraktion sowie pro NLP-Topic zu ermöglichen.
- (b) Implementieren Sie den Export objektorientiert, indem die einzelnen L^AT_EX-Bausteine mittels einer Methode *toTeX()* über Ihre Klassenstrukturen abgebildet und generiert werden können.
- (c) Die einzelnen Bausteine sind als Templates in der Datenbank abzubilden, welche ebenfalls über das Webinterface editierbar sind.

Die Generierung erfolgt hierbei über das Backend; das Frontend visualisiert das generierte PDF direkt innerhalb der Anwendung¹⁵, ohne vorherigen Download.

4 Dokumentation

10 Punkte

Dokumentieren Sie Ihren Quellcode und den Projektverlauf ausführlich und erstellen Sie als Gruppe eine Dokumentation:

- (a) Dokumentieren Sie den Quellcode ausführlich unter Verwendung von JavaDoc.
- (1) Dokumentieren Sie, welche Gruppenmitglieder welche Methoden implementiert haben.
 - (2) Dokumentieren Sie, welche Gruppenmitglieder welche Methoden modifiziert haben.
 - (3) Beschreiben Sie jede Methode ausführlich.
 - (4) Verwenden Sie `swagger.io` zur Dokumentation der REST-Methoden.
- (b) Erstellen Sie ein Benutzerhandbuch, in dem Sie die Nutzung Ihrer Applikation für Nutzende ausführlich erläutern.
- (c) Erstellen Sie eine Projektdokumentation und erläutern Sie basierend auf Ihrem Gantt-Diagramm den Projektverlauf. Dokumentieren Sie zudem die Aufgabenverteilung in der Gruppe sowie den Anteil der jeweiligen Eigenleistung.
- (d) Erstellen Sie eine L^AT_EX-basierte Präsentation, um Ihr Ergebnis im Abschlusstutorium vorzustellen.

Abgabe der Aufgabe

Ihr vollständiges, fertiges Projekt¹⁶ laden Sie bitte bis spätestens zum **22.03.2026** als ZIP-Archiv in OLAT hoch und **pushen** alles zusätzlich in Ihr GitLab-Projekt. Laden Sie das Projekt nur *einmal* pro Gruppe in OLAT hoch! Alle Abgaben, die nach diesem Datum hochgeladen werden, werden nicht berücksichtigt.

Ein Termin für die Präsentation der Applikation in den **Abschlusstutorien** wird von den Tutor*innen festgelegt.

¹⁵vgl. Overleaf

¹⁶Beinhaltet: Projektquellcode, Projektdokumentation (Quelle + PDF), Benutzerhandbuch.

Literatur

- Abrami, Giuseppe und Alexander Mehler (Aug. 2024). „Efficient, uniform and scalable parallel NLP pre-processing with DUUI: Perspectives and Best Practice for the Digital Humanities“. In: *Digital Humanities Conference 2024 - Book of Abstracts (DH 2024)*. Hrsg. von Jajwalya Karajgikar, Andrew Janco und Jessica Otis. DH. Washington, DC, USA: Zenodo, S. 15–18. DOI: [10.5281/zenodo.13761079](https://doi.org/10.5281/zenodo.13761079). URL: <https://doi.org/10.5281/zenodo.13761079>.
- Hammerla, Leon, Alexander Mehler und Giuseppe Abrami (2025). „Standardizing Heterogeneous Corpora with DUUR: A Dual Data- and Process-Oriented Approach to Enhancing NLP Pipeline Integration“. In: *Proceedings of the 14th International Joint Conference on Natural Language Processing and the 4th Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics*. Hrsg. von Kentaro Inui u. a. Mumbai, India: The Asian Federation of Natural Language Processing und The Association for Computational Linguistics, S. 1410–1425. ISBN: 979-8-89176-303-6. URL: <https://aclanthology.org/2025.findings-ijcnlp.87/>.
- Leonhardt, Alexander u. a. (2023). „Unlocking the Heterogeneous Landscape of Big Data NLP with DUUI“. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Hrsg. von Houda Bouamor, Juan Pino und Kalika Bali. Singapore: Association for Computational Linguistics, S. 385–399. URL: <https://aclanthology.org/2023.findings-emnlp.29>.

Anhang

DUUI-Komponenten

spaCy

Docker-Image: textimager-duui-spacy-single-de_core_news_sm:0.1.4

Remote-URL: <http://spacy.service.component.duui.texttechnologylab.org>

GerVader

Docker-Image: gervader_duui:1.0.2

Remote-URL: <http://gervader.service.component.duui.texttechnologylab.org>

ParlBert

Docker-Image: parlbert-topic-german:latest

Remote-URL: <http://parlbert.service.component.duui.texttechnologylab.org>

multilingual-sarcasm-detector

Docker-Image: duui-sarcasm-multilingual-sarcasm-detector:latest

Remote-URL: <http://sarcasm.service.component.duui.texttechnologylab.org>

WhisperX

Docker-Image: duui-whisperx:3.0.0-cuda

Remote-URL: <http://whisperx.service.component.duui.texttechnologylab.org>

Bitte beachten Sie, dass die Docker-Images unter Nutzung der URL: `docker.texttechnologylab.org` eingebunden werden müssen. Für die Verwendung von **GerVader** dem **multilingual-sarcasm-detector** müssen Sie noch einen Selektierungsparameter übergeben, Informationen hierzu finden Sie u.a. im [GitHub-Repository](#).